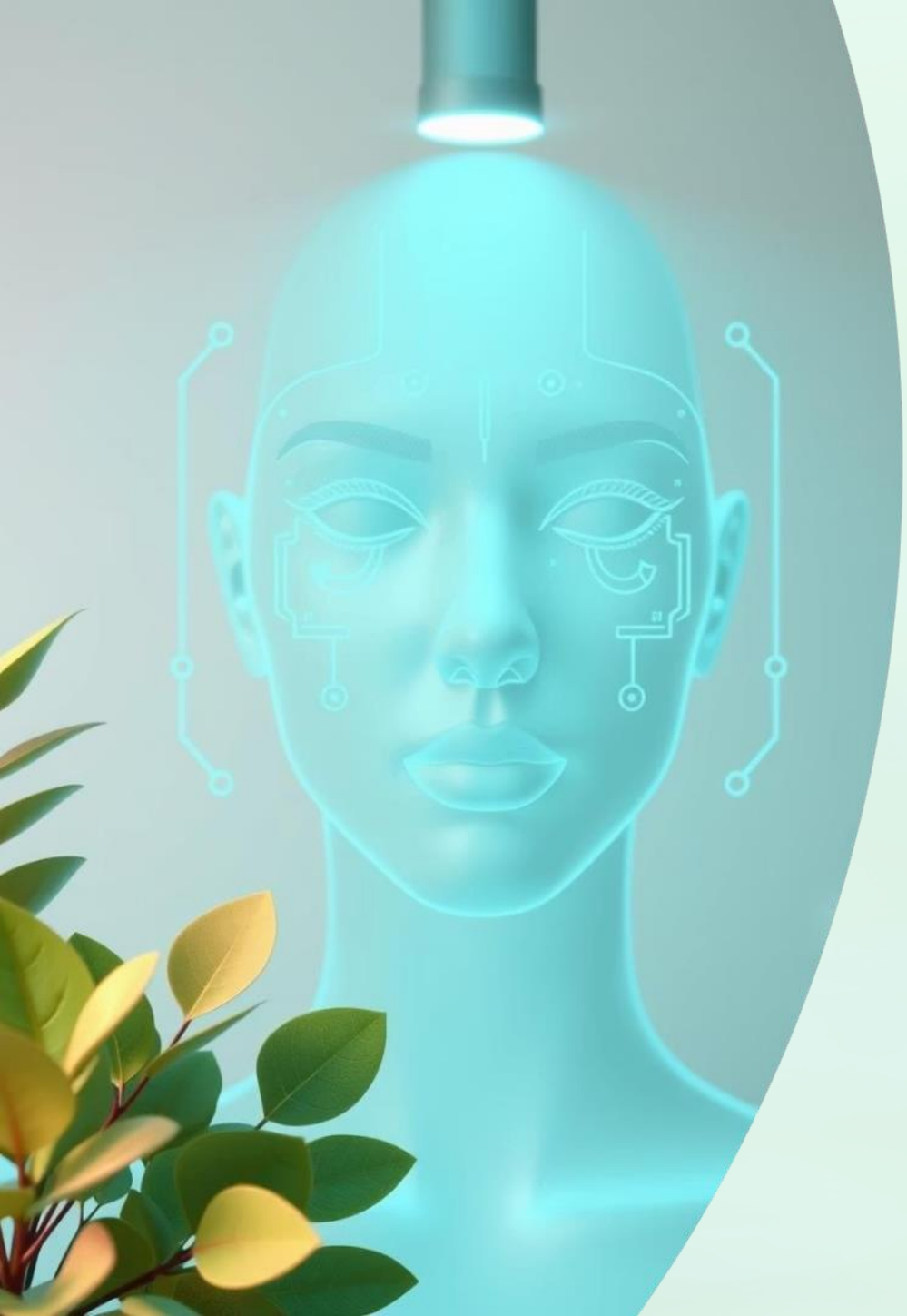


Facial Age Group Prediction and Identity Verification using Deep Learning

Using CNN-architecture

by AMAJALA RAVIKANTH
RA2412044015067

Project Overview

Face Detection

Uses Haar Cascade for accurate real-time real-time face detection.

Age Classification

Custom CNN predicts age groups: 18-25, 18-25, 26-35, 36-45, 46-60.

Live Processing

Processes video feed from camera instantaneously.

Dataset & Preprocessing

Dataset Source

CSV file with image paths and corresponding age labels.

Age Groups

- 18-25
- 26-35
- Ã ĊĎÆÇD
- 46-60

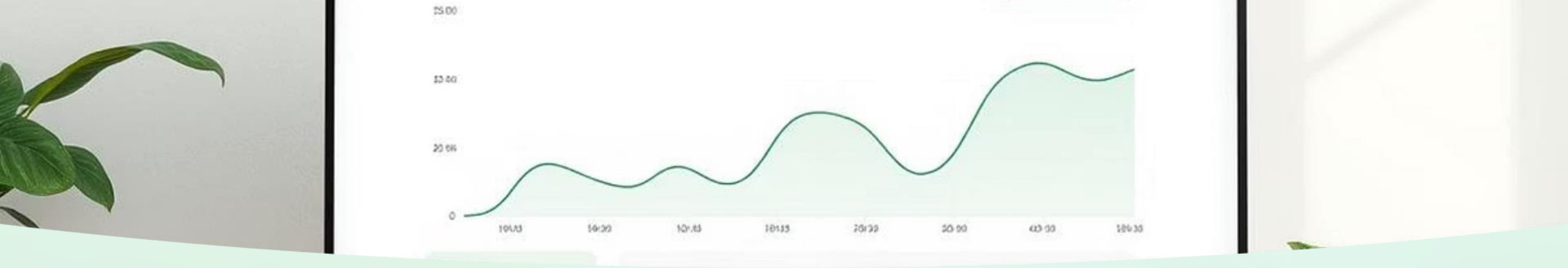
Image Processing

Resize images to 128x128 pixels and normalize pixel values.

Model Architecture

1. Conv2D(32)
2. MaxPool
3. Conv2D(64)
4. MaxPool
5. Flatten
6. Dense(128)
7. Dropout
8. Output(4)

Adam optimizer and categorical crossentropy loss for training.



Training Process

Epochs

ĈĆ ŅÞÕÕPØMŒŒŌŌŊ ŃŘŘÕÑCĔŎQÑØ ŇIMPMŒÑP

Validation Split

20% of data used for validation accuracy

Input Shape

Images resized to 128x128 with 3 color color channels

Output

Probability scores for four age groups groups

```
model.compile(optimizer='adam',
              loss='categorical_crossentropy')
```

Real-Time Detection Workflow

1. **Face Detection**
   Use Haar Cascade to find faces in video frame
2. **ROI Extraction**
   Crop and resize face regions to 128x128 pixels
3. **Age Prediction**
   Predict age group probabilities with CNN model
4. **Overlay Results**
   Display predicted age group labels on video feed

```
face_cascade.detectMultiScale(frame, ...)
```

Key Dependencies

OpenCV (cv2)

ĞVMÑÑ ŇÑPÑŃRÒŎŌ MŌŇÒÖMNJÑ ŐǾŎŃÑCŒŒŌŊ

TensorFlow/K Keras

Build and train CNN model

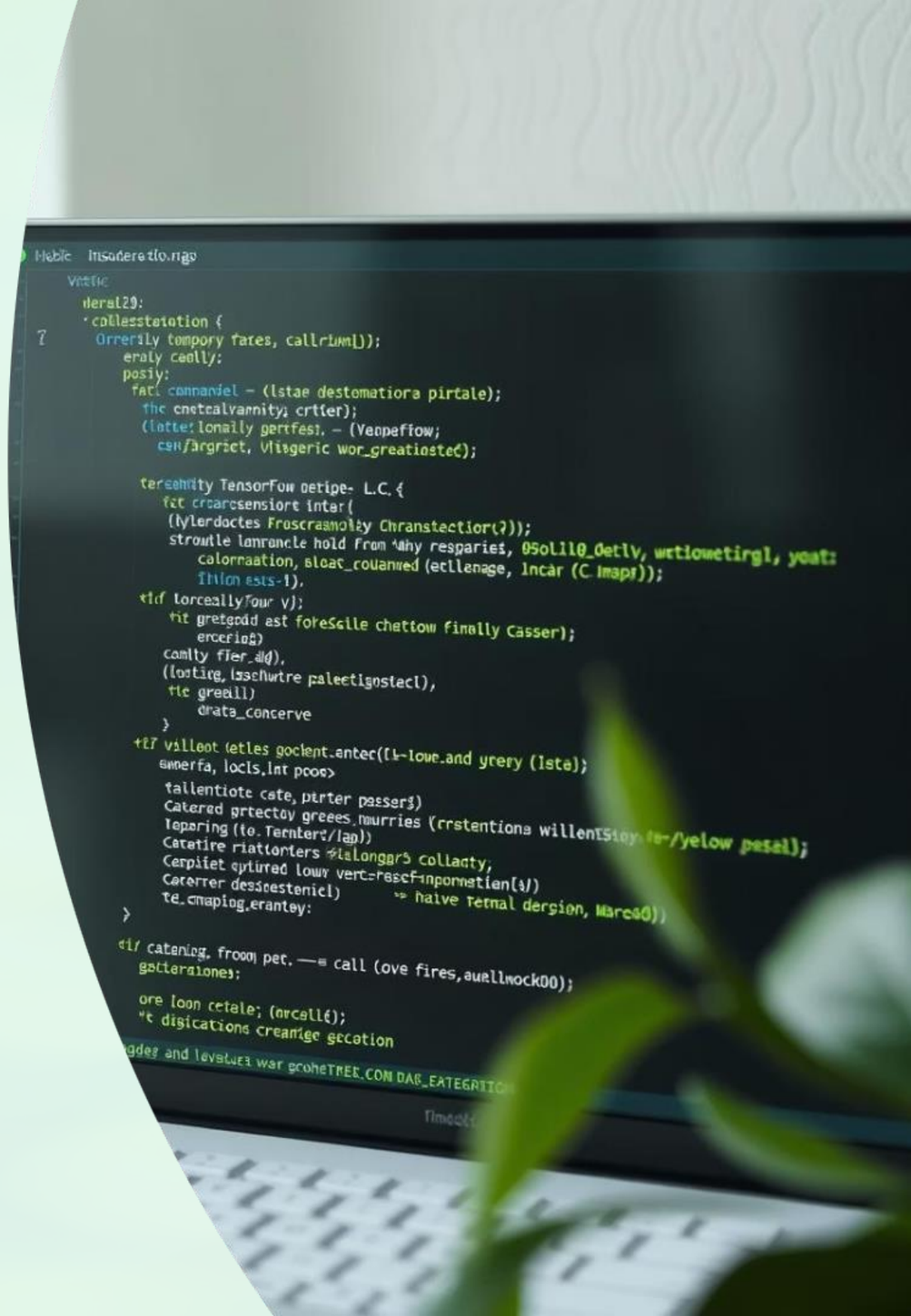
NumPy & Pandas

Data handling and preprocessing

scikit-learn

Data splitting and and evaluation tools

```
from tensorflow.keras.models import Sequential
```



Applications & Future Work

Use Cases

Retail analytics, security monitoring, customer demographics analysis

Improvements

- Higher-resolution CNN models
- Expanded and finer age ranges
- Include gender and demographic analysis

Future enhancements will increase accuracy and application scope.

```python
import cv2
import numpy as np
import pandas as pd
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from sklearn.preprocessing import LabelEncoder
import os

# ================================================================
# Configuration (Update these paths according to your system)
# ================================================================
DATASET_PATH = r'D:\SRM__DS AND AI\Semester-2\DEEP LEARNING APPROACHES\Mini Project\archive (1)'
CSV_FILE = 'age_detection.csv'
MODEL_SAVE_PATH = r'D:\Python\age_detection_model.h5'
LABEL_SAVE_PATH = r'D:\Python\age_group_classes.npy'

# ================================================================
# Step 1: Check for existing model or train new one
# ================================================================
def train_model():
    # Load dataset
    df = pd.read_csv(os.path.join(DATASET_PATH, CSV_FILE))

    # Preprocessing
    def convert_age(age_str):
        try: return int(age_str.split('-')[0])
        except: return np.nan

    df['age_num'] = df['age'].apply(convert_age).dropna().astype(int)
    df['age_group'] = df['age_num'].apply(lambda x: '18-25' if 18<=x<=25 else
                                          '26-35' if 26<=x<=35 else
                                          '36-45' if 36<=x<=45 else '46-60')

    le = LabelEncoder()
    df['label'] = le.fit_transform(df['age_group'])

    # Load images
    images = []
    labels = []
    for idx, row in df.iterrows():
        try:
            img_path = os.path.join(DATASET_PATH, row['file'])
            img = cv2.imread(img_path)
            if img is not None:
                img = cv2.resize(img, (128, 128))
                images.append(img)
                labels.append(row['label'])
        except:
```

File Edit Selection View Go Run Terminal Help

Python

EXPLORER

PYTHON

- .ipynb_checkpoints
- myenv
- age_detection_model.h5
- age_group_classes.npy
- Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2
- Facial_Age_Group_Prediction_and_Identity_Verification2.ipynb
- Handgesture_Volume_control.ipynb
- Handgesture_Volume_control.py
- hybrid_age_predictor.py 3
- requirements.txt

OUTLINE

TIMELINE

Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2 | hybrid_age_predictor.py 3

Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py > ...

```
20  def train_model():
47                  labels.append(row['label'])
48          except:
49              continue
50
51      # Convert to arrays
52      X = np.array(images) / 255.0
53      y = pd.get_dummies(labels).values
54
55      # Build model
56      model = Sequential([
57          Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)),
58          MaxPooling2D(2,2),
59          Conv2D(64, (3,3), activation='relu'),
60          MaxPooling2D(2,2),
61          Flatten(),
62          Dense(128, activation='relu'),
63          Dropout(0.5),
64          Dense(4, activation='softmax')
65      ])
66
67      model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
68      model.fit(X, y, epochs=10, validation_split=0.2)
69
70      # Save assets
71      model.save(MODEL_SAVE_PATH)
72      np.save(LABEL_SAVE_PATH, le.classes_)
73      return model, le.classes_
74
75  # ================================================================
76  # Step 2: Face detection and age prediction
77  # ================================================================
78  def main():
79      # Try to load existing model
80      try:
81          model = load_model(MODEL_SAVE_PATH)
82          age_groups = np.load(LABEL_SAVE_PATH)
83          print("Loaded pre-trained model")
84      except:
85          print("Training new model...")
86          model, age_groups = train_model()
87
88      # Initialize face detection
89      face_cascade = cv2.CascadeClassifier(
90          cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
91      )
92
93      cap = cv2.VideoCapture(0)
```

File Edit Selection View Go Run Terminal Help

Python

EXPLORER

PYTHON

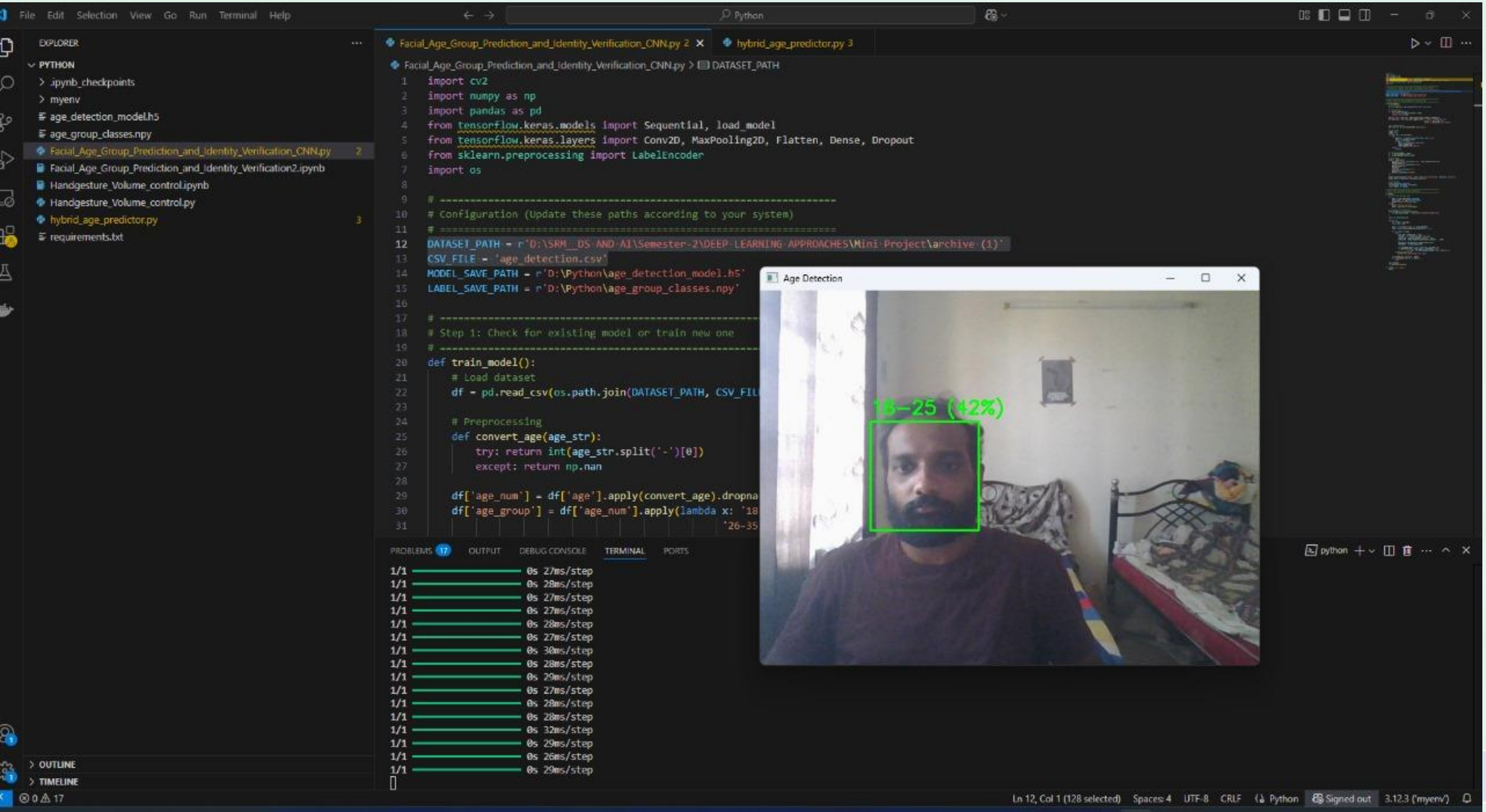
- .ipynb_checkpoints
- myenv
- age_detection_model.h5
- age_group_classes.npy
- Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2
- Facial_Age_Group_Prediction_and_Identity_Verification2.ipynb
- Handgesture_Volume_control.ipynb
- Handgesture_Volume_control.py
- hybrid_age_predictor.py 3
- requirements.txt

Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2 | hybrid_age_predictor.py 3

Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py > ...

```
78  def main():
94
95      while True:
96          ret, frame = cap.read()
97          if not ret: break
98
99          gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
100         faces = face_cascade.detectMultiScale(gray, 1.3, 5)
101
102         for (x,y,w,h) in faces:
103             try:
104                 face_img = frame[y:y+h, x:x+w]
105                 resized = cv2.resize(face_img, (128,128))
106                 normalized = resized.astype('float32') / 255.0
107                 prediction = model.predict(normalized[np.newaxis, ...])[0]
108
109                 age_group = age_groups[np.argmax(prediction)]
110                 confidence = np.max(prediction)
111
112                 cv2.rectangle(frame, (x,y), (x+w,y+h), (0,255,0), 2)
113                 cv2.putText(frame, f"{age_group} ({confidence:.0%})",
114                             (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0,255,0), 2)
115             except Exception as e:
116                 print(f"Face processing error: {e}")
117
118         cv2.imshow('Age Detection', frame)
119         if cv2.waitKey(1) & 0xFF == ord('q'):
120             break
121
122     cap.release()
123     cv2.destroyAllWindows()
124
125 if __name__ == "__main__":
126     main()
```

Result:


File Edit Selection View Go Run Terminal Help
Python
EXPLORER
PYTHON
.ipynb_checkpoints
myenv
age_detection_model.h5
age_group_classes.npy
Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2
Facial_Age_Group_Prediction_and_Identity_Verification2.ipynb
Handgesture_Volume_control.ipynb
Handgesture_Volume_control.py
hybrid_age_predictor.py 3
requirements.txt
Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2
hybrid_age_predictor.py 3
Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py > DATASET_PATH
import cv2
import numpy as np
import pandas as pd
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from sklearn.preprocessing import LabelEncoder
import os
# --
# Configuration (Update these paths according to your system)
# --
DATASET_PATH = r'D:\SRM_DS AND AI\Semester-2\DEEP LEARNING APPROACHES\Mini Project\archive (1)'
CSV_FILE = 'age_detection.csv'
MODEL_SAVE_PATH = r'D:\Python\age_detection_model.h5'
LABEL_SAVE_PATH = r'D:\Python\age_group_classes.npy'
# --
# Step 1: Check for existing model or train new one
# --
def train_model():
# Load dataset
df = pd.read_csv(os.path.join(DATASET_PATH, CSV_FIL
# Preprocessing
def convert_age(age_str):
try: return int(age_str.split('-')[0])
except: return np.nan
df['age_num'] = df['age'].apply(convert_age).dropna
df['age_group'] = df['age_num'].apply(lambda x: '18
'26-35
Age Detection
18–25 (42%)
PROBLEMS 17 OUTPUT DEBUG CONSOLE TERMINAL PORTS
python
1/1 0s 27ms/step
1/1 0s 28ms/step
1/1 0s 27ms/step
1/1 0s 27ms/step
1/1 0s 28ms/step
1/1 0s 27ms/step
1/1 0s 30ms/step
1/1 0s 28ms/step
1/1 0s 29ms/step
1/1 0s 27ms/step
1/1 0s 28ms/step
1/1 0s 28ms/step
1/1 0s 32ms/step
1/1 0s 29ms/step
1/1 0s 26ms/step
1/1 0s 29ms/step
OUTLINE
TIMELINE
Ln 12, Col 1 (128 selected) Spaces: 4 UTF-8 CRLF Python Signed out 3.12.3 ('myenv')

Facial Age Group Prediction and Identity Verification using Deep Learning

Hybrid CNN-Transformer architecture leveraging EfficientNetB0 and MTCNN for real-time age regression and classification.

Problem Statement

Goals

- Predict exact age (regression)
- Classify age groups (classification)

Applications

- Ã Ï ÑŃÞØÒPŘMŌŇCÐÞØQÑÒŌŌMŌŃŃÑ
- Marketing analytics
- Age-restricted content filtering

Dataset and Preprocessing

Dataset Details

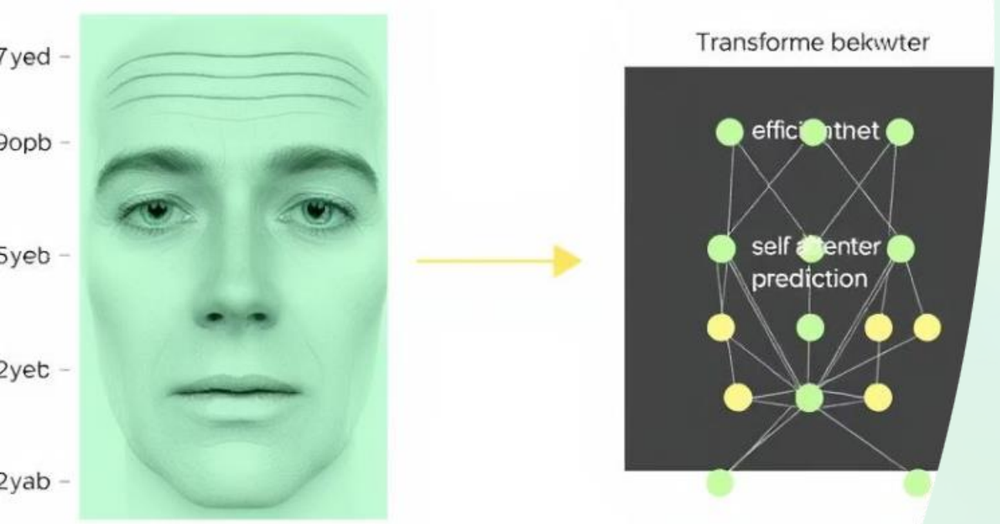
Custom CSV with image paths and precise age labels

- Age groups: 0–17, 18–29, 30–49, 50+
- Label encoding for classification

Preprocessing Steps

- Image resizing to EfficientNet input size
- Standard EfficientNet preprocessing
- Face detection using MTCNN at inference

Model Architecture Overview



EfficientNetB0 Backbone Backbone

Pretrained, weights frozen for feature extraction

Transformer Block

ÎØÖÑÑÑÈÉÑÈÈÈÉÓVÞÒVÕ ØÑÕVÞÖÖÕÈÈÒÖÖÑÈÈÖÖÕ F Í Í ŅÑVPÞØÑÈ

Dual Output Heads

- Age Regression with linear output
- Age Group Classification with softmax

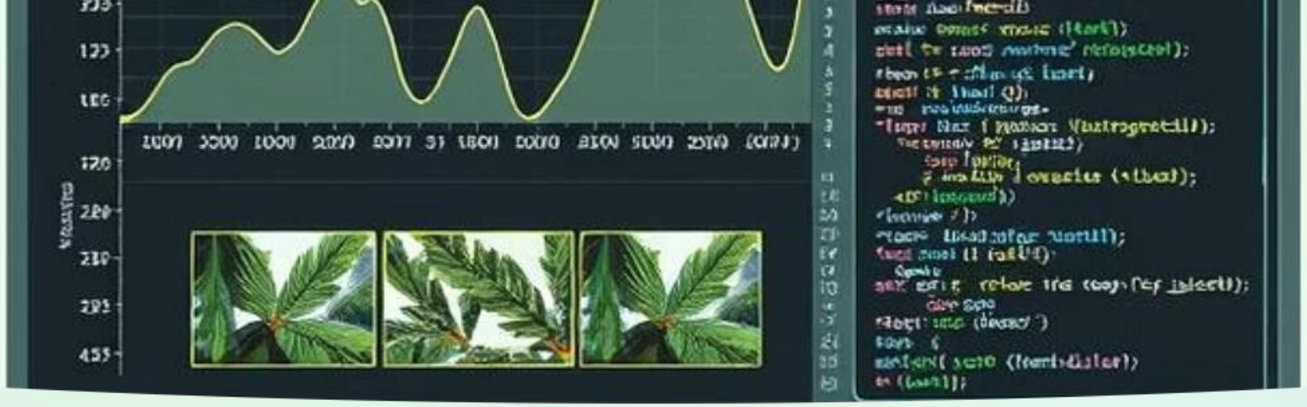
Transformer Block Details

Key Components

- Multi-Head Attention captures feature dependencies
- Feedforward Dense Layers use GELU activation
- Ã IMŘÑØÍ ÒØÖMÕÕRVPÒÒÒ ÑŌCEÞØÑCEPØMÒÒÒÒNJCEPMVNÒÒÕPŘ
- Dropout mitigates overfitting

Role in Model

Models cross-spatial context from CNN-extracted features



Training Pipeline

1

Data Augmentation

- Rotation, zoom, shifts, flips

2

Training Details

Ã ĊĆ ÑŐŎŃŒERÒPO Ė ŇIMÖ ŎŐPŎŎÒŖÑØ

- Huber loss for regression
- Categorical cross-entropy for classification
- EarlyStopping and ReduceLROnPlateau callbacks

3

Model Saving

Model weights (.h5) and class labels (.npy)

Real-Time Prediction Pipeline Pipeline

MTCNN Face Detection

GÑPÑŃPMŌŇ ŃØŎŐŅVŃÑCEŅØŎÖ RÑNŃMÖ ÒŌŐÞP

Inference Steps

- Resize and preprocess cropped face
- Predict exact age and class probabilities
- Overlay predictions on video frame

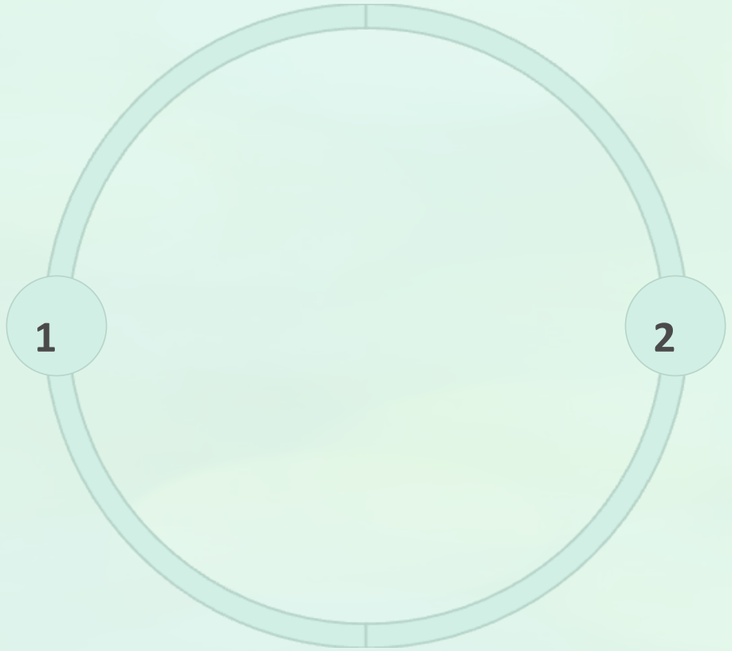
Live Demo

Webcam feed runs continuously, press 'q' to exit

Results & Conclusion

Performance

- Accurate age regression output
- Reliable age group classification



Key Findings

- Hybrid EfficientNet + Transformer excels excels
- Improved contextual feature understanding
- Real-time system feasible with optimizations

File Edit Selection View Go Run Terminal Help

Python

EXPLORER

PYTHON

- .ipynb_checkpoints
- myenv
- age_detection_model.h5
- age_group_classes.npy
- Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2
- Facial_Age_Group_Prediction_and_Identity_Verification2.ipynb
- Handgesture_Volume_control.ipynb
- Handgesture_Volume_control.py
- hybrid_age_predictor.py 3
- requirements.txt

Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2 | hybrid_age_predictor.py 3

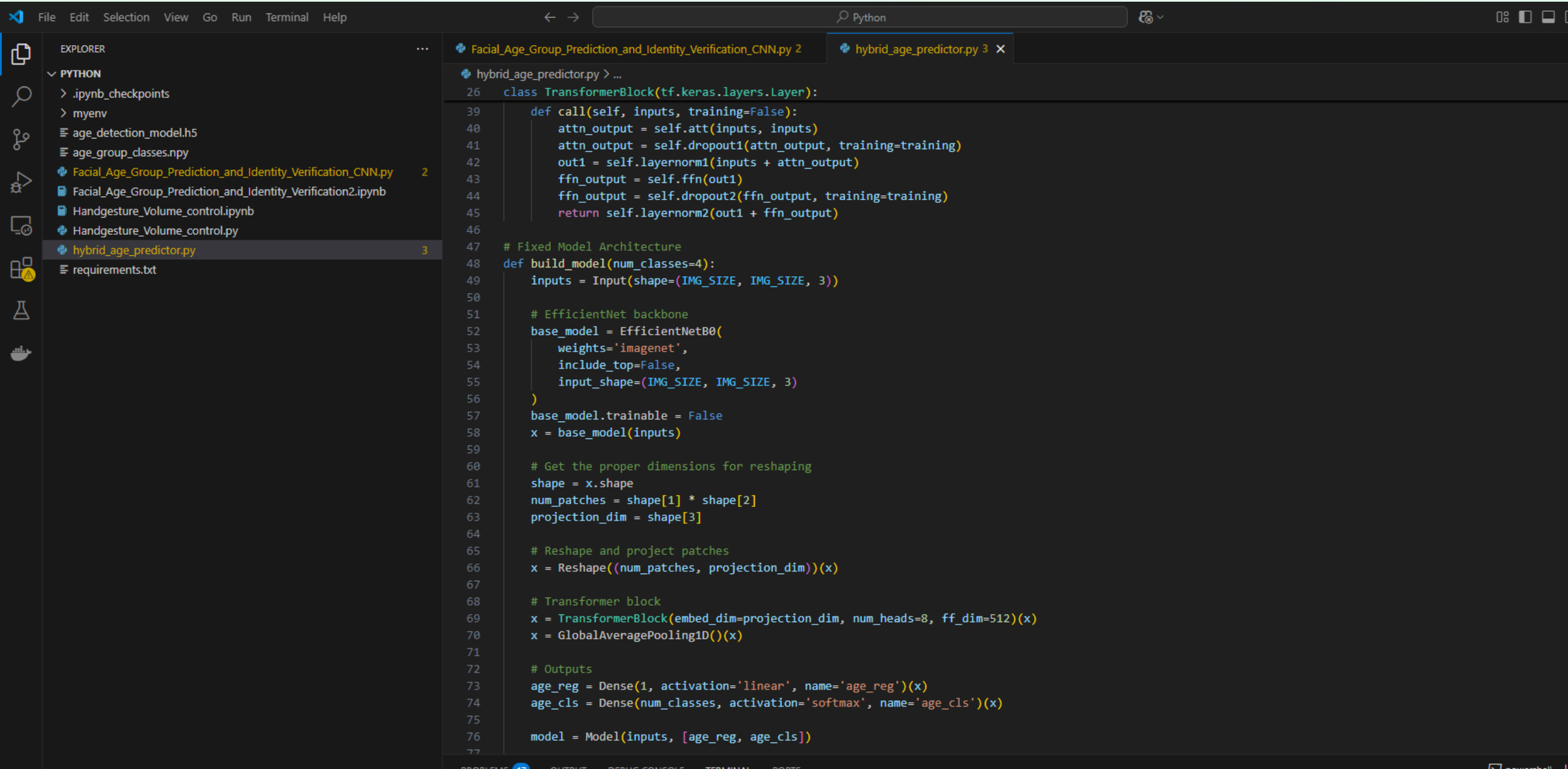
hybrid_age_predictor.py > ...

```python
import cv2
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras.layers import (Input, Dense, Dropout, GlobalAveragePooling1D,
                                     MultiHeadAttention, LayerNormalization, Conv2D,
                                     Reshape, Flatten)
from tensorflow.keras.models import Model
from tensorflow.keras.applications import EfficientNetB0
from mtcnn import MTCNN
from sklearn.preprocessing import LabelEncoder
import os
import warnings
warnings.filterwarnings('ignore')

# Configuration
DATASET_PATH = r'D:\SRM__DS AND AI\Semester-2\DEEP LEARNING APPROACHES\Mini Project\archive (1)'
CSV_FILE = 'age_detection.csv'
MODEL_SAVE_PATH = r'D:\Python\age_prediction_model.h5'
CLASSES_SAVE_PATH = r'D:\Python\age_classes.npy'
IMG_SIZE = 224
BATCH_SIZE = 32
EPOCHS = 30

# Fixed Transformer Block
class TransformerBlock(tf.keras.layers.Layer):
    def __init__(self, embed_dim, num_heads, ff_dim, rate=0.1):
        super().__init__()
        self.att = MultiHeadAttention(num_heads=num_heads, key_dim=embed_dim)
        self.ffn = tf.keras.Sequential([
            Dense(ff_dim, activation="gelu"),
            Dense(embed_dim)
        ])
        self.layernorm1 = LayerNormalization(epsilon=1e-6)
        self.layernorm2 = LayerNormalization(epsilon=1e-6)
        self.dropout1 = Dropout(rate)
        self.dropout2 = Dropout(rate)

    def call(self, inputs, training=False):
        attn_output = self.att(inputs, inputs)
```


File Edit Selection View Go Run Terminal Help
Python
EXPLORER
PYTHON
.ipynb_checkpoints
myenv
age_detection_model.h5
age_group_classes.npy
Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2
Facial_Age_Group_Prediction_and_Identity_Verification2.ipynb
Handgesture_Volume_control.ipynb
Handgesture_Volume_control.py
hybrid_age_predictor.py 3
requirements.txt
Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2
hybrid_age_predictor.py 3
hybrid_age_predictor.py > ...
class TransformerBlock(tf.keras.layers.Layer):
def call(self, inputs, training=False):
attn_output = self.att(inputs, inputs)
attn_output = self.dropout1(attn_output, training=training)
out1 = self.layernorm1(inputs + attn_output)
ffn_output = self.ffn(out1)
ffn_output = self.dropout2(ffn_output, training=training)
return self.layernorm2(out1 + ffn_output)
# Fixed Model Architecture
def build_model(num_classes=4):
inputs = Input(shape=(IMG_SIZE, IMG_SIZE, 3))
# EfficientNet backbone
base_model = EfficientNetB0(
weights='imagenet',
include_top=False,
input_shape=(IMG_SIZE, IMG_SIZE, 3)
)
base_model.trainable = False
x = base_model(inputs)
# Get the proper dimensions for reshaping
shape = x.shape
num_patches = shape[1] * shape[2]
projection_dim = shape[3]
# Reshape and project patches
x = Reshape((num_patches, projection_dim))(x)
# Transformer block
x = TransformerBlock(embed_dim=projection_dim, num_heads=8, ff_dim=512)(x)
x = GlobalAveragePooling1D()(x)
# Outputs
age_reg = Dense(1, activation='linear', name='age_reg')(x)
age_cls = Dense(num_classes, activation='softmax', name='age_cls')(x)
model = Model(inputs, [age_reg, age_cls])

File Edit Selection View Go Run Terminal Help

Python

EXPLORER

PYTHON

- .ipynb_checkpoints
- myenv
- age_detection_model.h5
- age_group_classes.npy
- Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2
- Facial_Age_Group_Prediction_and_Identity_Verification2.ipynb
- Handgesture_Volume_control.ipynb
- Handgesture_Volume_control.py
- hybrid_age_predictor.py 3
- requirements.txt

Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2 | hybrid_age_predictor.py 3

hybrid_age_predictor.py > ...

```
48  def build_model(num_classes=4):
76      model = Model(inputs, [age_reg, age_cls])
77
78      model.compile(
79          optimizer=tf.keras.optimizers.Adam(3e-5),
80          loss={'age_reg': 'huber_loss', 'age_cls': 'categorical_crossentropy'},
81          metrics={'age_cls': 'accuracy'},
82          loss_weights={'age_reg': 0.4, 'age_cls': 0.6}
83      )
84
85      return model
86
87  # Get Age Group
88  def get_age_group(age):
89      if age < 18:
90          return '0-17'
91      elif age < 30:
92          return '18-29'
93      elif age < 50:
94          return '30-49'
95      else:
96          return '50+'
97
98  # Enhanced Data Loading with detailed debugging
99  def load_data():
100     try:
101         csv_path = os.path.join(DATASET_PATH, CSV_FILE)
102         if not os.path.exists(csv_path):
103             raise FileNotFoundError(f"CSV file not found at: {csv_path}")
104
105         df = pd.read_csv(csv_path)
106         print(f"\nLoaded CSV with {len(df)} rows. First 5 entries:")
107         print(df.head())
108
109         # Convert age to numeric and clean data
110         df['age'] = pd.to_numeric(df['age'], errors='coerce')
111         df = df.dropna(subset=['age'])
112         df['age'] = df['age'].astype(int)
113         df['age_group'] = df['age'].apply(get_age_group)
```

File Edit Selection View Go Run Terminal Help

Python

EXPLORER

PYTHON

- .ipynb_checkpoints
- myenv
- age_detection_model.h5
- age_group_classes.npy
- Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2
- Facial_Age_Group_Prediction_and_Identity_Verification2.ipynb
- Handgesture_Volume_control.ipynb
- Handgesture_Volume_control.py
- hybrid_age_predictor.py 3
- requirements.txt

Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2 | hybrid_age_predictor.py 3

hybrid_age_predictor.py > ...

```
99  def load_data():
113         df['age_group'] = df['age'].apply(get_age_group)
114
115         le = LabelEncoder()
116         df['group_label'] = le.fit_transform(df['age_group'])
117
118         images, age_labels, group_labels = [], [], []
119         missing_images = 0
120         failed_images = 0
121
122         print("\nStarting image loading...")
123         for idx, row in df.iterrows():
124             # Handle different path formats
125             img_path = os.path.join(DATASET_PATH, row['image'])
126
127             # Try alternative path joining if first attempt fails
128             if not os.path.exists(img_path):
129                 alt_path = os.path.join(DATASET_PATH, 'images', row['image'])
130                 if os.path.exists(alt_path):
131                     img_path = alt_path
132                 else:
133                     print(f"Row {idx}: Image not found at {img_path}")
134                     missing_images += 1
135                     continue
136
137             try:
138                 img = cv2.imread(img_path)
139                 if img is None:
140                     print(f"Row {idx}: Failed to load (may be corrupt) - {img_path}")
141                     failed_images += 1
142                     continue
143
144                 img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
145                 img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)  # Ensure RGB format
146                 img = tf.keras.applications.efficientnet.preprocess_input(img)
147
148                 images.append(img)
149                 age_labels.append(row['age'])
150                 group_labels.append(row['group_label'])
```

File Edit Selection View Go Run Terminal Help

Python

EXPLORER

PYTHON

- .ipynb_checkpoints
- myenv
- age_detection_model.h5
- age_group_classes.npy
- Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2
- Facial_Age_Group_Prediction_and_Identity_Verification2.ipynb
- Handgesture_Volume_control.ipynb
- Handgesture_Volume_control.py
- hybrid_age_predictor.py 3
- requirements.txt

Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2 | hybrid_age_predictor.py 3

hybrid_age_predictor.py > ...

```
99  def load_data():
152                 if idx % 100 == 0:
153                     print(f"Processed {idx+1}/{len(df)} images")
154
155             except Exception as e:
156                 print(f"Row {idx}: Error processing image - {str(e)}")
157                 failed_images += 1
158
159         print("\nImage loading summary:")
160         print(f"Total CSV rows: {len(df)}")
161         print(f"Successfully loaded images: {len(images)}")
162         print(f"Missing images: {missing_images}")
163         print(f"Failed/corrupt images: {failed_images}")
164
165         if len(images) == 0:
166             raise ValueError("No valid images were loaded. Check your dataset paths and integrity.")
167
168         X = np.array(images)
169         y_reg = np.array(age_labels)
170         y_cls = tf.keras.utils.to_categorical(group_labels, num_classes=len(le.classes_))
171
172         return X, y_reg, y_cls, le.classes_
173
174     except Exception as e:
175         print(f"\nError in load_data(): {str(e)}")
176         raise
177
178 # Enhanced Training Pipeline
179 def train_model():
180     try:
181         print("\nStarting training process...")
182         X, y_reg, y_cls, classes = load_data()
183
184         # Data augmentation
185         datagen = tf.keras.preprocessing.image.ImageDataGenerator(
186             rotation_range=20,
187             width_shift_range=0.2,
188             height_shift_range=0.2,
189             zoom_range=0.2,
190             horizontal_flip=True,
```

File Edit Selection View Go Run Terminal Help

Python

EXPLORER

PYTHON

- .ipynb_checkpoints
- myenv
- age_detection_model.h5
- age_group_classes.npy
- Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2
- Facial_Age_Group_Prediction_and_Identity_Verification2.ipynb
- Handgesture_Volume_control.ipynb
- Handgesture_Volume_control.py
- hybrid_age_predictor.py 3
- requirements.txt

Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2 | hybrid_age_predictor.py 3

hybrid_age_predictor.py > ...

```
179  def train_model():
192      )
193
194      # Create training and validation generators
195      train_generator = datagen.flow(
196          X,
197          {'age_reg': y_reg, 'age_cls': y_cls},
198          batch_size=BATCH_SIZE,
199          subset='training'
200      )
201
202      validation_generator = datagen.flow(
203          X,
204          {'age_reg': y_reg, 'age_cls': y_cls},
205          batch_size=BATCH_SIZE,
206          subset='validation'
207      )
208
209      model = build_model(num_classes=len(classes))
210      print("\nModel summary:")
211      model.summary()
212
213      print("\nTraining model...")
214      history = model.fit(
215          train_generator,
216          epochs=EPOCHS,
217          validation_data=validation_generator,
218          callbacks=[
219              tf.keras.callbacks.ReduceLROnPlateau(monitor='val_loss', patience=3),
220              tf.keras.callbacks.EarlyStopping(
221                  monitor='val_age_cls_accuracy',
222                  patience=5,
223                  restore_best_weights=True
224              ),
225              tf.keras.callbacks.ModelCheckpoint(
226                  MODEL_SAVE_PATH,
227                  save_best_only=True,
228                  monitor='val_age_cls_accuracy'
229              )
```

PROBLEMS 17 OUTPUT DEBUG CONSOLE TERMINAL PORTS

powershell

File Edit Selection View Go Run Terminal Help

Python

EXPLORER

PYTHON

- .ipynb_checkpoints
- myenv
- age_detection_model.h5
- age_group_classes.npy
- Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2
- Facial_Age_Group_Prediction_and_Identity_Verification2.ipynb
- Handgesture_Volume_control.ipynb
- Handgesture_Volume_control.py
- hybrid_age_predictor.py 3
- requirements.txt

Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2 | hybrid_age_predictor.py 3

hybrid_age_predictor.py > ...

```python
def train_model():
                    monitor='val_age_cls_accuracy'
                )
            ],
            verbose=1
        )

        # Save model and classes
        model.save(MODEL_SAVE_PATH)
        np.save(CLASSES_SAVE_PATH, classes)
        print(f"\nModel successfully saved to {MODEL_SAVE_PATH}")
        print(f"Class labels saved to {CLASSES_SAVE_PATH}")

        return model, classes

    except Exception as e:
        print(f"\nError in train_model(): {str(e)}")
        raise

# Enhanced Prediction Class
class AgePredictor:
    def __init__(self, force_train=False):
        self.detector = MTCNN()

        if force_train or not os.path.exists(MODEL_SAVE_PATH):
            print("\nModel not found or forced training - starting training...")
            self.model, self.classes = train_model()
        else:
            try:
                print("\nLoading existing model...")
                self.model = tf.keras.models.load_model(
                    MODEL_SAVE_PATH,
                    custom_objects={'TransformerBlock': TransformerBlock}
                )
                self.classes = np.load(CLASSES_SAVE_PATH)
                print("Model loaded successfully")
            except Exception as e:
                print(f"Error loading model: {str(e)}")
                print("Falling back to training new model...")
                self.model, self.classes = train_model()
```

File Edit Selection View Go Run Terminal Help

Python

EXPLORER

PYTHON

- .ipynb_checkpoints
- myenv
- age_detection_model.h5
- age_group_classes.npy
- Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2
- Facial_Age_Group_Prediction_and_Identity_Verification2.ipynb
- Handgesture_Volume_control.ipynb
- Handgesture_Volume_control.py
- hybrid_age_predictor.py 3
- requirements.txt

Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2 | hybrid_age_predictor.py 3

hybrid_age_predictor.py > ...

```
247  class AgePredictor:
248      def __init__(self, force_train=False):
266              self.model, self.classes = train_model()
267
268      def predict(self, frame):
269          try:
270              # Convert to RGB and detect faces
271              rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
272              results = self.detector.detect_faces(rgb_frame)
273
274              if not results:
275                  return frame
276
277              # Get largest face
278              main_face = max(results, key=lambda x: x['box'][2] * x['box'][3])
279              x, y, w, h = main_face['box']
280
281              # Ensure coordinates are within frame bounds
282              x, y = max(0, x), max(0, y)
283              w, h = min(w, frame.shape[1] - x), min(h, frame.shape[0] - y)
284
285              # Extract and preprocess face
286              face = rgb_frame[y:y+h, x:x+w]
287              face = cv2.resize(face, (IMG_SIZE, IMG_SIZE))
288              face = tf.keras.applications.efficientnet.preprocess_input(face)
289
290              # Make prediction
291              age, group = self.model.predict(face[np.newaxis, ...], verbose=0)
292              predicted_age = max(0, int(age[0][0]))  # Ensure age isn't negative
293              age_group = self.classes[np.argmax(group[0])]
294              confidence = np.max(group[0])
295
296              # Draw results on frame
297              cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
298              text = f"Age: {predicted_age} ({age_group}, {confidence:.0%})"
299              cv2.putText(frame, text, (x, y - 10),
300                          cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)
301
302          except Exception as e:
303              print(f"Prediction error: {str(e)}")
```

PROBLEMS 17 OUTPUT DEBUG CONSOLE TERMINAL PORTS

File Edit Selection View Go Run Terminal Help

Python

EXPLORER

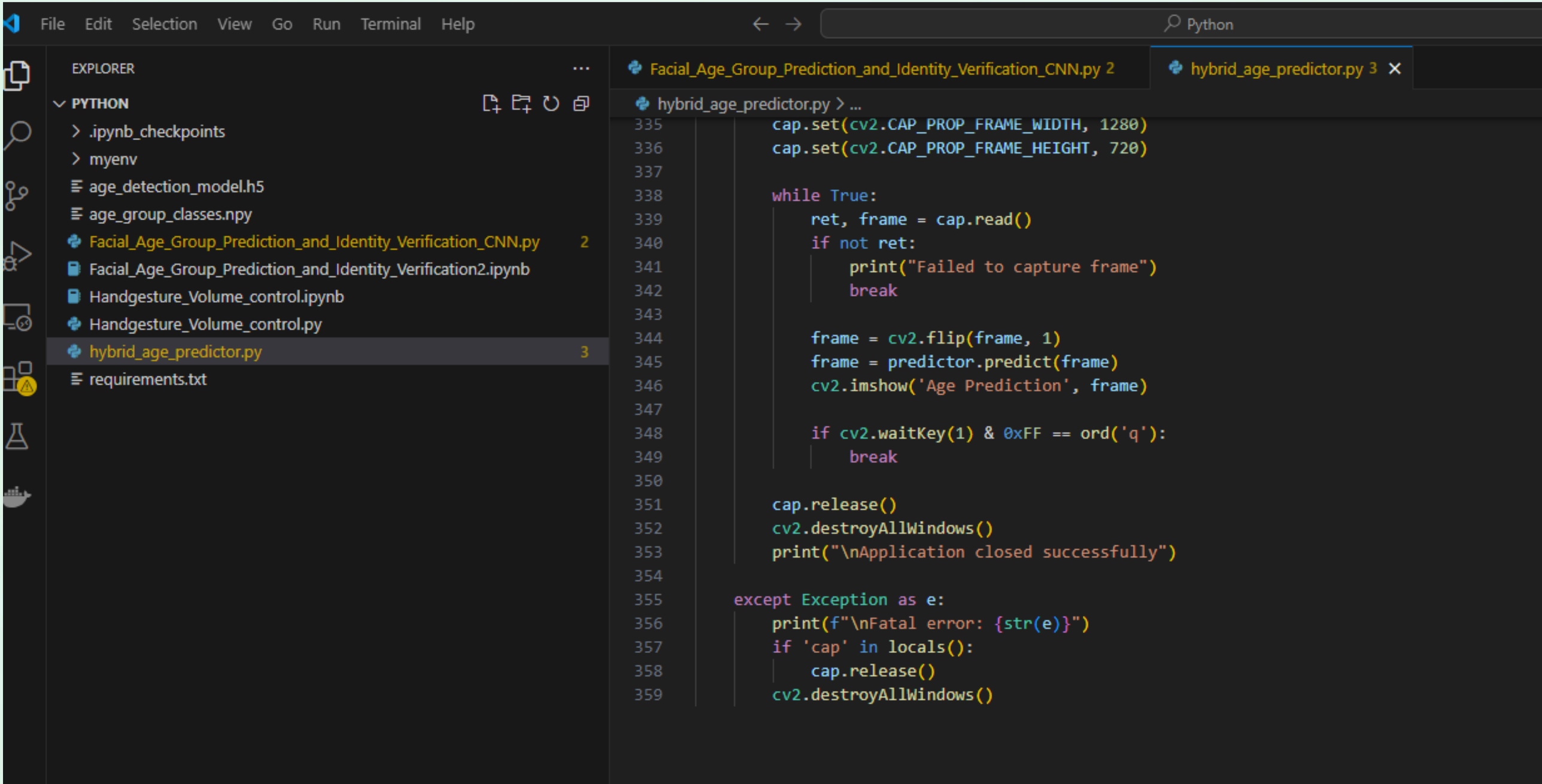
PYTHON

- .ipynb_checkpoints
- myenv
- age_detection_model.h5
- age_group_classes.npy
- Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2
- Facial_Age_Group_Prediction_and_Identity_Verification2.ipynb
- Handgesture_Volume_control.ipynb
- Handgesture_Volume_control.py
- hybrid_age_predictor.py 3
- requirements.txt

Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2 | hybrid_age_predictor.py 3

hybrid_age_predictor.py > ...

```
247  class AgePredictor:
305          return frame
306
307  # Main Execution with better error handling
308  if __name__ == "__main__":
309      try:
310          print("\nInitializing Age Prediction System...")
311
312          # First test dataset structure
313          print("\nTesting dataset structure...")
314          csv_path = os.path.join(DATASET_PATH, CSV_FILE)
315          if os.path.exists(csv_path):
316              df = pd.read_csv(csv_path)
317              test_img_path = os.path.join(DATASET_PATH, df.iloc[0]['image'])
318              print(f"Sample image path: {test_img_path}")
319              print(f"Exists: {os.path.exists(test_img_path)}")
320          else:
321              print(f"CSV file not found at {csv_path}")
322
323          # Initialize predictor
324          predictor = AgePredictor(force_train=False)
325
326          # Start real-time prediction
327          print("\nStarting real-time prediction...")
328          print("Press 'q' to quit")
329
330          cap = cv2.VideoCapture(0)
331          if not cap.isOpened():
332              raise RuntimeError("Could not open video capture")
333
334          # Set camera resolution
335          cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
336          cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)
337
338          while True:
339              ret, frame = cap.read()
340              if not ret:
341                  print("Failed to capture frame")
342                  break
```


File Edit Selection View Go Run Terminal Help
Python
EXPLORER
PYTHON
.ipynb_checkpoints
myenv
age_detection_model.h5
age_group_classes.npy
Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2
Facial_Age_Group_Prediction_and_Identity_Verification2.ipynb
Handgesture_Volume_control.ipynb
Handgesture_Volume_control.py
hybrid_age_predictor.py 3
requirements.txt
Facial_Age_Group_Prediction_and_Identity_Verification_CNN.py 2
hybrid_age_predictor.py 3
hybrid_age_predictor.py > ...
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)
while True:
ret, frame = cap.read()
if not ret:
print("Failed to capture frame")
break
frame = cv2.flip(frame, 1)
frame = predictor.predict(frame)
cv2.imshow('Age Prediction', frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
break
cap.release()
cv2.destroyAllWindows()
print("\nApplication closed successfully")
except Exception as e:
print(f"\nFatal error: {str(e)}")
if 'cap' in locals():
cap.release()
cv2.destroyAllWindows()

Conclusion: Power of Hybrid CNN-Transformer Architecture

- Hybrid model excels in facial age & identity tasks
- CNN captures fine-grained local features effectively
- Transformer models global dependencies for context
- Improved accuracy vs. standalone CNNs
- Robustness in real-time prediction scenarios